ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

EVALUATING PROGRAM COMPREHENSION THROUGH EXPLAINABLE DEEP LEARNING: A MULTI-LEVEL FRAMEWORK FOR IDENTIFIER READABILITY, CODE SNIPPET ANALYSIS, AND DEVELOPER EXPERIENCE CLASSIFICATION

Synopsis submitted to Alliance University

for the Award of

DOCTOR OF PHILOSOPHY

In

Faculty of Engineering and Technology

By

BHARAT BABASO MANE

Reg No: 20030145CSE003

Under the Supervision

Of

DR. RATHNAKAR ACHARY

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advance Computing

Alliance University

Central Campus, Chikkahadage Cross Chandapura-Anekal, Main Road, Bengaluru, Karnataka 562106

2026

Contents

1.ABSTRACT

The synopsis is meant to be a detailed summary of the Ph.D. dissertation work with important results highlighting the original contributions by the candidate. It should give a general outline of the thesis. The literature survey and review of previous research work should be brief with a limited purpose of highlighting the important contributions.

Program comprehension - the act of understanding what a piece of software does, how it is structured, and who built it - underpins nearly every activity in software engineering. Maintenance, code review, onboarding, defect prediction, and refactoring all begin with a developer reading and making sense of source code. The cost of poor comprehension is well documented: maintenance alone accounts for roughly seventy percent of total software lifecycle expenditure, and a significant portion of that cost traces directly to code that is difficult to read.

Despite this, automated tools for measuring and explaining code quality have remained narrow. Most assess surface characteristics - line length, indentation depth, bracket density - while ignoring the semantic content that experienced developers actually attend to when judging code. Identifier names, which carry the developer's conceptual model of the solution, receive almost no attention. The experience of the developer who wrote the code, which shapes the quality of every decision recorded in it, is measured even less.

This thesis addresses the problem from three complementary directions, each operating at a different level of abstraction. The first study, IRAF-XADL, examines identifier names: it asks whether individual function names, parameter names, and variable names are readable, using ten linguistically and cognitively grounded features combined with CodeBERT embeddings and a Self-Attention BiLSTM classifier. The second study, ECRVR-MVEL, widens the lens to whole code snippets: it predicts snippet-level readability by combining three deep classifiers - a Graph Convolutional Network, a Deep Belief Network, and a Bidirectional Temporal Convolutional Network - in a weighted majority vote. The third study, EESQA-DELMOA, shifts focus from the code to the person who wrote it: it assesses developer experience level using bio-inspired feature selection and a Simplified Spiking Neural Network, on the premise that developer expertise is a strong predictor of the software quality they produce.
All three studies share a commitment to explainability. IRAF-XADL uses SHAP; ECRVR-MVEL uses LIME; EESQA-DELMOA exposes feature importance through its selection stage. Together they form a layered picture of software quality that no single-level analysis could provide.

Experimental results, validated on publicly available benchmark datasets, demonstrate state-of-the-art performance at each level: 97.36% test accuracy for identifier readability in Python (97.94% for C++), 98.15% test accuracy for snippet-level comprehension in Python (98.38% for C++), and 98.74% test accuracy for developer-level quality assessment - with an execution time of 8.27 seconds, the lowest among all compared methods.

2. INTRODUCTION

Code is read far more often than it is written. A developer working on an established codebase may spend the majority of their working time reading - tracing control flow, identifying the purpose of a function, inferring the intent behind a variable name. This activity, called program comprehension, is the cognitive foundation on which all maintenance, debugging, and extension work rests. When code is hard to comprehend, everything built on top of it costs more.

The quality of source code - the property that makes it either easy or difficult to comprehend - is multidimensional. At the finest grain, it resides in the names chosen for individual identifiers: functions, parameters, classes, and variables. A name like `calculateTotalAmountForUser` communicates intent immediately; a name like `calc` or `tmp2` does not. At a broader grain, quality is visible in the structure of a code snippet: whether it avoids unnecessary complexity, whether its logic is expressed clearly. At the broadest grain, quality reflects the experience and discipline of the developer who wrote the code. Each level contributes to the whole. Each level has historically been studied in isolation.

The present research is motivated by two observations. First, existing automated readability tools measure proxies rather than substance. Metrics based on line length, operator count, and indentation capture something about visual presentation, but miss the semantic content - the naming quality, the conceptual clarity - that drives actual comprehension. Second, no existing framework assesses readability at multiple levels simultaneously while also explaining its decisions. In practice, a software engineering team needs to know not just whether a codebase is readable, but which identifiers are weak, why, and which developers may need support or training. Explanations are not a luxury; they are what makes an automated tool usable.

This thesis proposes three studies that together address this gap. The studies are ordered by the scale of their unit of analysis: identifier, code snippet, developer. They share a dataset for the first two studies (the Code Snippets: Insights and Readability dataset from Kaggle), use an independent dataset for the third (a developer-experience dataset from Zenodo), and consistently integrate explainability components so that every prediction is accompanied by a human-interpretable account of what drove it.

2.1 Background

Identifier naming has been studied since the 1970s, largely in the context of naming conventions and style guides. The insight that naming quality correlates with software quality dates at least to the work of Lawrie et al. (2006) and Binkley et al. (2009), who showed that better identifier names reduce the time developers need to understand code. More recently, machine learning approaches have been applied to predict identifier quality, but most rely on a small set of handcrafted features and binary classification schemes.

Code readability at the snippet level has attracted attention since Buse and Weimer (2010) introduced their annotated dataset and showed that simple linear models trained on whitespace features could predict human readability judgements. Deep learning approaches followed, achieving progressively better accuracy, but explainability remained an afterthought.

Developer experience as a predictor of software quality has been explored through proxy measures - commit count, project tenure, bug-fix rate - but few studies directly classify developer experience level from code and then link that classification to quality outcomes. The gap is particularly notable given how much resource allocation decisions (project assignment, code review pairing) depend

implicitly on experience judgements.

2.2 Problem Statement

Automated tools for assessing source code quality have remained narrow in scope. Most measure surface properties such as line length, indentation, and operator count, while ignoring the semantic content that experienced developers actually attend to, particularly the quality of identifier names. No existing system assesses code comprehensibility at more than one level of abstraction, and none provides interpretable explanations alongside its predictions. This thesis addresses the absence of a unified, explainable framework that can evaluate program comprehension at the identifier level, the code snippet level, and the developer experience level within a single coherent methodology.

3. LITERATURE SURVEY

A representative review of related work is presented here, organized by the three levels addressed in the thesis. The review is intentionally selective, highlighting the contributions most directly relevant to each study rather than providing exhaustive coverage.

3. Literature Review Summary

Sr. No.	Author(s) / Year	Journal / Source	Research Area	Major Contribution	Research Gap Identified	Relevance to Present Research
A. Identifier Naming and Identifier Readability						
1	Feng et al., 2020	Findings of EMNLP	Code representation learning	Introduced CodeBERT, a pre-trained model for programming language and natural language representation, useful for code search, summarization and code understanding tasks.	CodeBERT provides strong semantic embeddings, but it does not directly solve identifier readability assessment, code snippet readability classification or developer-experience-based software quality assessment.	Used as the core semantic representation model in Study 1 and Study 2 for capturing contextual meaning from identifiers and code snippets.
2	Butler, Wermelinger, Yu and Sharp, 2010	European Conference on Software Maintenance and Reengineering	Identifier names and code quality	Demonstrated that poor identifier names and naming anti-patterns are associated with lower code quality and higher defect-proneness.	Existing analysis was based mainly on naming rules and empirical observations; semantic, contextual and cognitive dimensions of identifiers were not deeply modelled.	Provides the foundation for the proposed ten-dimensional identifier readability feature set in IRAF-XADL.
3	Lawrie, Morrell, Field and Binkley, 2006	IEEE International Conference on Program Comprehension	Identifier naming and program comprehension	Established that meaningful and descriptive identifier names improve program comprehension and help developers understand source	The study focused mainly on human comprehension and naming practices, but did not provide an automated deep learning-based identifier readability assessment model.	Supports the need for Study 1, which evaluates identifier readability using linguistically

				code more effectively.		grounded features, CodeBERT embeddings and explainable deep learning.
B. Code Readability, Understandability and Explainable AI						
4	Oliveira et al., 2025	IEEE Transactions on Software Engineering	Code understandability improvement in code reviews	Studied how code understandability can be improved through code review activities and identified practical aspects influencing understandability.	The work focused on understandability improvement in review contexts, but did not propose an automated multi-level explainable framework for readability and software quality assessment.	Supports the practical importance of automated explainable program comprehension models for maintainability and code review support.
5	Scalabrino et al., 2016/2018/ 2019	Software Engineering Research Publications	Code readability and understandability	Extended code readability research by considering semantic textual features, identifier terms and human-perceived understandability.	Although semantic features were considered, the approaches did not provide a robust multi-class deep ensemble model with local interpretability for code snippet readability.	Supports the design of ECRVR-MVEL, which uses CodeBERT representations, deep classifiers and LIME-based explanation.
6	Lundberg and Lee, 2017	Advances in Neural Information Processing Systems	Explainable Artificial Intelligence	Proposed SHAP, a theoretically grounded feature attribution method based on Shapley values for explaining model predictions.	SHAP was not specifically applied to identifier readability assessment with language-specific preprocessing and deep attention-based models.	Used in Study 1 to explain the contribution of identifier tokens and readability parameters in IRAF-XADL.
7	Ribeiro, Singh and Guestrin, 2016	ACM SIGKDD	Explainable Artificial Intelligence	Proposed LIME, a model-agnostic explanation technique that explains	LIME was not sufficiently explored for explaining deep ensemble-based code readability	Used in Study 2 to provide local interpretability for

				individual predictions using local interpretable surrogate models.	classification decisions.	ECRVR-MVEL predictions.
8	Buse and Weimer, 2010	IEEE Transactions on Software Engineering	Code readability prediction	Proposed one of the earliest machine learning-based code readability metrics using human-labelled readability data and lexical/structural features.	The model relied heavily on handcrafted surface-level features such as line length, indentation and textual metrics, with limited semantic understanding and explainability.	Motivates the need for deep contextual code representation and explainable readability classification in Study 2.
C. Developer Experience and Software Quality Assessment						
9	Brasil-Silva and Siqueira, 2022	ACM/SIGAPP Symposium on Applied Computing	Developer experience metrics	Presented a systematic mapping of metrics used to quantify software developer experience.	The study mapped developer-experience metrics but did not develop an optimized deep learning classifier for developer experience level classification.	Supports Study 3, where developer experience is classified using optimized feature selection and deep learning.
10	Yadav, Singh and Suri, 2019	Information and Software Technology	Developer expertise and bug triaging	Proposed methods for ranking developers based on expertise score for bug triaging and software maintenance support.	The work focused mainly on developer ranking for bug assignment and did not address developer-centric software quality assessment using metaheuristic optimization and neural classification.	Provides background for Study 3, which classifies developer experience levels and links them to software quality assessment.

Summary of Research Gap

The reviewed literature shows that identifier readability, code snippet readability and developer experience have generally been studied as separate problems. Earlier readability models mainly relied on handcrafted surface-level features and traditional machine learning methods. Although recent approaches use deep learning and code embeddings, limited work has focused on explainability, multi-class readability classification, or multi-level software quality assessment. Similarly, developer

experience has been studied using proxy metrics such as commit count, project activity and expertise ranking, but there is limited work on optimized deep learning-based classification of developer experience levels for software quality assessment.
Therefore, the present research addresses this gap by proposing a multi-level explainable framework covering: (i) identifier readability assessment using CodeBERT, Self-Attention BiLSTM and SHAP; (ii) code snippet readability classification using CodeBERT, weighted majority voting ensemble learning and LIME; and (iii) developer-centric software quality assessment using BAHB-based feature selection, Simplified Spiking Neural Network classification and AMBOA-based hyperparameter optimization.

3.1 Identifier Readability and Code Quality

Lawrie, Morrell, Field, and Binkley (2007) established empirically that longer, more descriptive identifiers improve program comprehension speed, demonstrating that full-word names produce substantially better outcomes than abbreviations or single letters across accuracy, recall, and task completion measures. Hofmeister et al. (2017) extended these findings in a controlled experiment with 72 professional developers, showing that descriptive names enabled 19% faster bug detection - a margin with direct practical significance for code review and maintenance productivity. Schankin et al. (2011) used eye-tracking to confirm that developers fixate longer on cryptic identifiers, with the fixation overhead scaling with naming ambiguity and correlating directly with increased cognitive load. Butler et al. (2010, 2019) catalogued identifier naming antipatterns - single-letter names, non-dictionary tokens, inconsistent casing - and demonstrated through repository mining that modules exhibiting more such flaws carry statistically higher defect densities. Arnaoudova et al. (2016) formalised a taxonomy of linguistic antipatterns (misleading names, homonyms, ambiguous names) and showed that developers uniformly perceive these as obstacles to comprehension, frequently triggering renaming refactorings in version history.

Machine learning approaches to identifier quality began with decision trees and support vector machines trained on lexical features. Rahman et al. (2019) used word embeddings to capture semantic content, improving over lexical baselines. Transformer-based models such as CodeBERT (Feng et al., 2020) have since provided 768-dimensional contextual representations of code tokens, capturing variable dependencies and programming-specific language patterns across six programming languages - but they have not been applied to identifier readability with an attention-augmented recurrent classifier that jointly exploits contextual embeddings and linguistically grounded features prior to this thesis.

SHAP (Lundberg and Lee, 2017) provides a theoretically grounded framework for feature attribution based on Shapley values from cooperative game theory. Its application to identifier readability classification with a Self-Attention BiLSTM - where attribution must be distributed across ten purpose-designed readability parameters - is introduced in this thesis.

3.2 Code Readability Prediction

Buse and Weimer (2010) created the first human-annotated code readability dataset and trained a logistic regression model on whitespace and lexical features, establishing the baseline that later work has sought to surpass. Scalabrino et al. (2016, 2018) extended this with a larger annotation study that incorporated semantic textual features - informative terms in identifiers and comments - and showed these features substantially improve correlation with human judgements of understandability.

Scalabrino et al. (2019) subsequently trained a machine learning classifier for code understandability on crowd-sourced annotations, confirming that automated readability assessment is feasible and that naming-related features are among the strongest predictors.

Deep learning approaches have progressively supplanted handcrafted-feature models. Mi et al. (2025) applied graph convolutional networks to code dependency structures for readability classification, reporting 92.87% accuracy on the Python dataset used in this thesis - a strong single-classifier result that our weighted ensemble surpasses by more than five percentage points. Deep belief networks learn hierarchical probabilistic representations of code features through stacked Restricted Boltzmann Machines; bidirectional temporal convolutional networks capture long-range sequential dependencies in both causal directions via dilated convolutions. These three architectures are structurally diverse - one relational, one probabilistic, one sequential - making them natural candidates for an ensemble whose diversity reduces prediction variance. Ensemble methods combining structurally distinct classifiers via weighted majority voting have improved robustness in malware classification and software defect prediction, but their application to code readability with learned, per-classifier weights remains unexplored prior to this thesis.

LIME (Ribeiro et al., 2016) generates local, model-agnostic explanations by fitting an interpretable linear surrogate in the neighbourhood of each prediction. While SHAP has been applied to some code tasks, LIME's application to snippet-level readability with a weighted ensemble of deep classifiers is novel in this thesis.

3.3 Developer Experience and Software Quality

Developer experience has long been studied through observable proxy measures - commit frequency, project tenure, bug-fix rate, code ownership - used as inputs to defect prediction and developer recommendation systems. Zamir et al. (2025) demonstrated that integrating pull-request commentary with developer profile features can recommend the most suitable reviewer for a task with high precision, but their approach relied on handcrafted heuristics rather than a formal deep-learning classification framework. Akhtar and Daviglus (2025) showed that AI-assisted analysis of behavioural coding patterns - debugging sessions, review participation, edit frequency - can distinguish developer proficiency levels more reliably than traditional metrics, but without classifying experience into discrete, actionable categories. Perez, Urtado, and Vauttier (2023) addressed this gap by releasing the developer-experience dataset used in this thesis: 703 profiles across six classes (Experienced Software Engineer, Software Architect, Software Engineer, Non-Software Engineer, Bot, Unknown) with 26 quantitative features derived from observable open-source contribution activity, providing the first publicly available benchmark for formal developer experience classification.

Spiking neural networks (SNNs) process information as discrete temporal spike events rather than continuous activations, making them inherently suited to modelling temporal activity patterns such as developer contribution frequency over time. SNNs have been applied in biomedical signal processing and anomaly detection; the Simplified Spiking Neural Network (SSNN) variant retains the core leaky integrate-and-fire dynamics at reduced computational cost, demonstrating 43% lower execution time than equivalent artificial neural networks in comparative benchmarks. No prior work has applied SNNs to developer experience classification in a software engineering context.

The Bio-inspired Artificial Hummingbird Behaviour (BAHB) algorithm and the Adaptive Migration Butterfly Optimisation Algorithm (AMBOA) are recent metaheuristic methods that balance global exploration with local exploitation through biologically motivated movement strategies. BAHB has demonstrated strong performance in high-dimensional feature selection by simulating three foraging

behaviours (guided search, territorial search, migration); AMBOA has shown effective hyperparameter tuning via linearly decaying inertia weights. Neither algorithm has been applied to developer experience classification, and no prior work combines SSNN with both BAHB feature selection and AMBOA tuning for software quality assessment.

4. SCOPE OF THE THESIS

This thesis is scoped to three related but distinct studies, each of which addresses program comprehension at a specific level of abstraction. The scope boundaries are as follows.

The first study (IRAF-XADL) is limited to Python and C++ source code. Identifier extraction uses language-specific abstract syntax tree parsers - LibCST for Python, Tree-Sitter for C++ - and is therefore restricted to these languages. The ten readability features are linguistically grounded but reflect natural English; the model's performance on code written in other natural languages has not been evaluated.

The second study (ECRVR-MVEL) operates on the same Python and C++ dataset and is therefore subject to the same language scope. Snippet boundaries are defined by the dataset; the model does not segment code into snippets autonomously.

The third study (EESQA-DELMOA) is applied to a developer experience dataset drawn from open-source contributions. It classifies developers into six experience categories as defined by the dataset's labelling scheme. It does not assess the quality of specific commits or pull requests, and it does not attempt to predict future performance.

All three studies use publicly available benchmark datasets and compare against published baselines. The thesis does not include user studies or evaluation with industrial code; those are identified as directions for future work.

5. OBJECTIVES

This thesis has three objectives:

1. To assess identifier readability in Python and C++ source code by extracting ten naming quality features through AST-based parsing, combining them with CodeBERT embeddings in a Self-Attention BiLSTM classifier, and using SHAP to explain what drives each prediction.
2. To predict code snippet readability by building an ensemble of three structurally different classifiers — GCN, DBN and Bi-TCN — combined through weighted majority voting, with LIME explanations showing which parts of the code influenced the result.
3. To classify developer experience level from observable activity features by selecting the most relevant features using bio-inspired optimisation, training a Simplified Spiking Neural Network, and evaluating the system against published baselines on accuracy and execution time.

6. RESEARCH METHODOLOGY

6.1 Study 1: IRAF-XADL - Identifier Readability Analysis Framework using Explainable Attention-Based Deep Learning

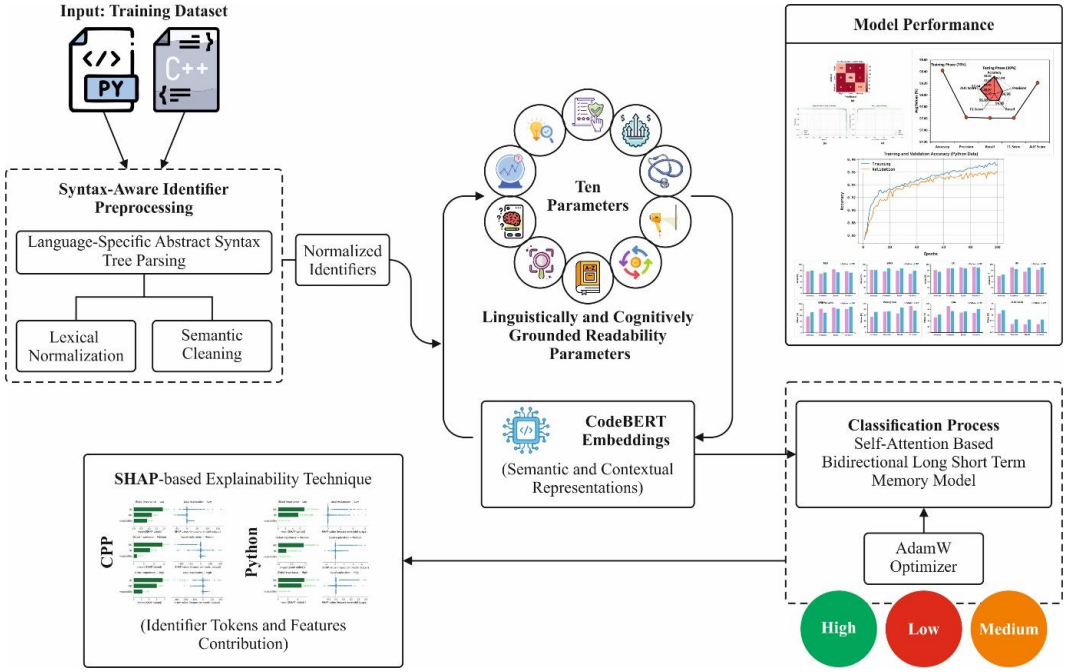

Figure 1. IRAF-XADL framework: from raw Python/C++ source through identifier preprocessing, ten-parameter extraction, CodeBERT embeddings, and SA-BiLSTM classification to SHAP explainability.

Stage 1
Identifier extraction and normalisation. Identifiers are extracted from source code using language-specific abstract syntax tree parsers: LibCST for Python and Tree-Sitter for C++. This ensures structurally accurate extraction of function names, parameter names, class names, and variable names. The extracted identifiers undergo lexical normalisation: camelCase and snake_case splitting, digit-letter separation, lowercasing, and lemmatisation using WordNet. Generic programming stopwords (var, obj, tmp) and English function words are removed.

Stage 2
Ten readability parameters. From each normalised identifier, ten features are computed:
(1) Meaningful Clarity (MC) - proportion of tokens that are recognisable English words;
(2) Naming Conformance (NC) - adherence to language-specific naming conventions such as snake_case for Python functions;
(3) Optimal Length (OL) - a Gaussian score peaking within an empirically defined ideal length range;
(4) Domain Relevance (DR) - fraction of tokens matching domain-specific vocabulary inferred from the surrounding snippet;
(5) Pronounceability (PR) - vowel ratio relative to an English language baseline;
(6) Lexical Familiarity (LF) - average token frequency in natural language corpora;
(7) Context Consistency (CC) - embedding similarity between an identifier and its peers;
(8) Scope Appropriateness (SA) - whether identifier length is proportionate to the variable's scope;
(9) Cognitive Load Score (CLS) - a composite of MC, LF, and an ambiguity penalty;

(10) Predictability (PRED) - how likely an identifier's tokens are given its neighbouring identifiers.

Stage 3
CodeBERT embeddings. Each identifier is encoded using CodeBERT (microsoft/codebert-base), a 12-layer Transformer pre-trained on six programming languages. The per-token embeddings are mean-pooled to produce a 768-dimensional fixed-length representation per identifier.

Stage 4
SA-BiLSTM classifier. The CodeBERT embeddings and the ten readability features are concatenated and fed as a sequence to a Self-Attention BiLSTM. The BiLSTM processes the identifier sequence bidirectionally across three layers with 128 hidden units each and dropout 0.3. A multi-head self-attention mechanism (four heads, dimension 128) then weights the BiLSTM hidden states to produce a context vector. A dense layer of 64 units with ReLU activation produces the final three-class prediction: High, Medium, or Low readability.

Stage 5
AdamW optimisation. The model is trained for 100 epochs with batch size 32. AdamW decouples weight decay ($\lambda = 0.01$) from the gradient update, improving generalisation. Learning rate is 0.001; $\beta_1 = 0.9$, $\beta_2 = 0.999$; gradient clipping at 1.0.

Stage 6
SHAP explainability. SHAP KernelExplainer computes Shapley values for each of the ten readability features. Global summary plots reveal which features most influence predictions across the dataset; local dot plots show feature contributions for individual identifiers.

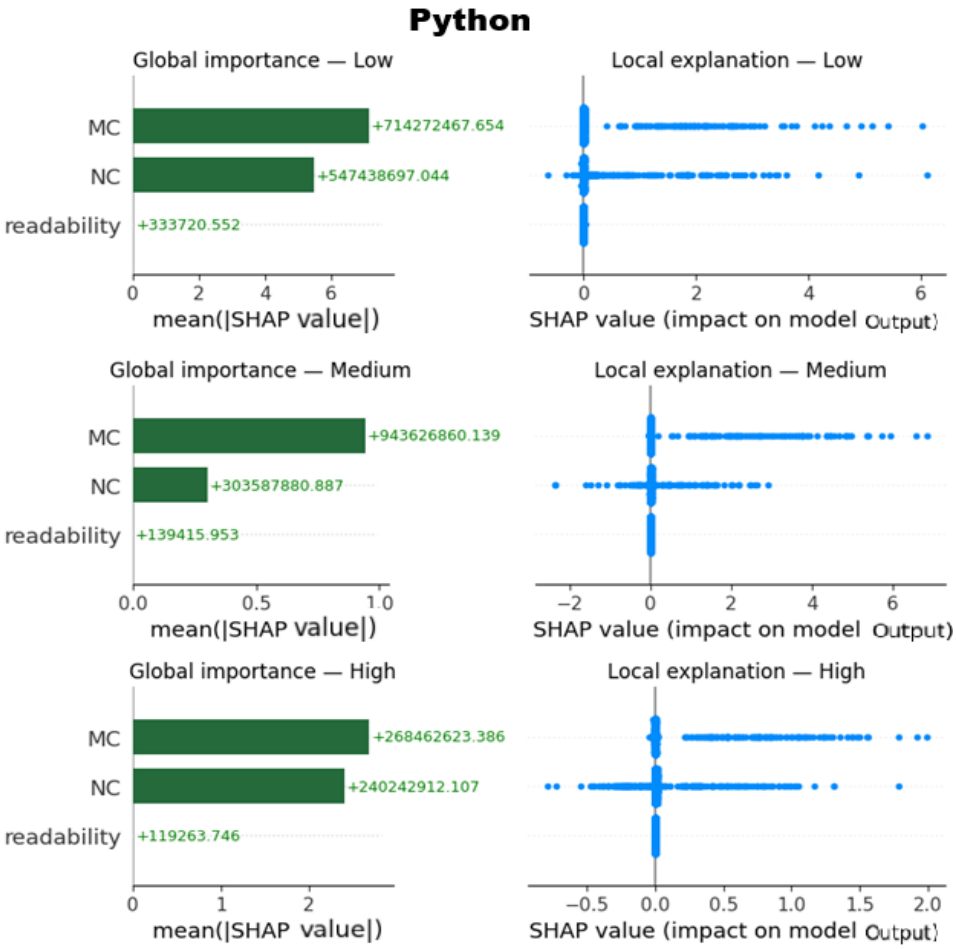

Figure 2. SHAP global and local explanations for the Python dataset (Study 1): Meaningful Clarity (MC) and Naming Conformance (NC) dominate across all three readability classes.

6.2 Study 2: ECRVR-MVEL - Explainable Code Readability Classification Using Vector Representations and Majority Voting-Based Ensemble Learning

The input is a full code snippet; the output is one of three readability classes (High, Medium, Low) with a LIME explanation.

Stage 1

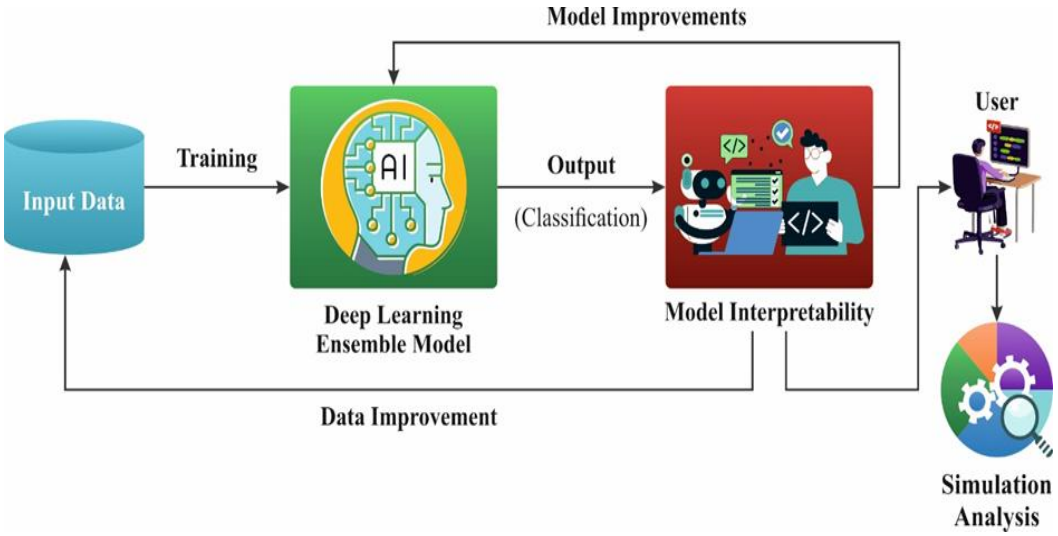

Figure 3. ECRVR-MVEL high-level workflow: input data passes through the deep learning ensemble, model interpretability (LIME), and feeds back for improvement.

Text preprocessing. Four sequential operations prepare the raw source code. (1) Tokenisation decomposes each snippet into atomic tokens - keywords, identifiers, literals, operators, and delimiters - using language-specific tokenisers that preserve the syntactic integrity of both C++ and Python inputs. (2) Comment removal and whitespace normalisation strips single-line and multi-line comments and normalises irregular indentation and whitespace to reduce vocabulary noise; indentation is handled carefully for Python to preserve syntactic structure. (3) Language detection automatically identifies the input language via file extension, reserved keyword analysis, and syntactic pattern matching, routing each snippet to the appropriate tokenisation pipeline. (4) Sequence encoding maps the tokenised sequence to numerical representations using a vocabulary derived from the training corpus, with padding or truncation to a uniform maximum length.

Stage 2
CodeBERT embeddings. CodeBERT (microsoft/codebert-base) provides the dense vector backbone. The [CLS] token embedding from the 12th transformer layer yields a 768-dimensional fixed-length snippet representation. For snippets exceeding CodeBERT's 512-token limit, a sliding window strategy encodes overlapping windows and averages the resulting [CLS] embeddings, with positional encoding preserved within each window to maintain sequential dependencies. The resulting embedding captures variable dependencies, control-flow patterns, and cross-statement semantic relationships through the model's multi-head self-attention mechanism.

Stage 3
Graph Convolutional Network (GCN). The GCN models structural dependency relationships within the code snippet. Node embeddings are initialised from the CodeBERT output and updated through three GCN layers, each aggregating information from neighbouring nodes weighted by cosine similarity between node representations. Each layer expands the receptive field by one hop, capturing structural dependencies up to three steps away. Global mean pooling aggregates the final node embeddings into a graph-level vector for classification, producing a representation sensitive to architectural relationships rather than only the surface token sequence.

Stage 4
Deep Belief Network (DBN). The DBN provides a hierarchical probabilistic feature extractor through stacked Restricted Boltzmann Machine (RBM) layers trained greedily in an unsupervised phase. Each RBM models the joint distribution over its visible and hidden units via an energy function

parameterised by the layer weights; the hidden activations of each layer serve as visible inputs to the next. The hierarchical architecture progressively abstracts higher-level representations, allowing the DBN to capture complex probabilistic patterns in the code embedding space that discriminate between readability classes.

Stage 5
Bidirectional Temporal Convolutional Network (Bi-TCN). The Bi-TCN models sequential token dependencies in both causal directions. Each bidirectional temporal block applies forward dilated convolutions (capturing past context) and backward dilated convolutions (incorporating future context), with exponentially growing dilation factors that extend the receptive field without increasing parameter count proportionally. A feature fusion layer merges the forward and backward outputs into a unified temporal representation. Residual connections in each block stabilise gradient flow through the deep network, preventing vanishing gradients and preserving important lower-level features.

Stage 6
Weighted majority voting + Nadam optimisation. The GCN, DBN, and Bi-TCN each produce a probability distribution over the three readability classes. These are combined via weighted majority voting, where per-classifier weights are learned during training to proportionally upweight classifiers with higher historical validation performance. Nadam (Nesterov-accelerated Adaptive Moment Estimation) optimises all three classifiers by computing gradients at the lookahead position after applying Nesterov momentum - anticipating the next parameter state - enabling faster and more responsive convergence compared to standard Adam in non-convex loss landscapes.

Stage 7
LIME explainability. LIME generates a per-prediction explanation by perturbing the input snippet in the feature space, querying the ensemble, and fitting an interpretable linear surrogate to the sampled responses. The surrogate's coefficients identify the code tokens and structural features most influential in the readability verdict, with positive coefficients indicating features that push toward High readability and negative coefficients those that push toward Low. Feature importance scores are normalised and displayed per prediction, providing transparent, human-auditable justification for every classification decision.

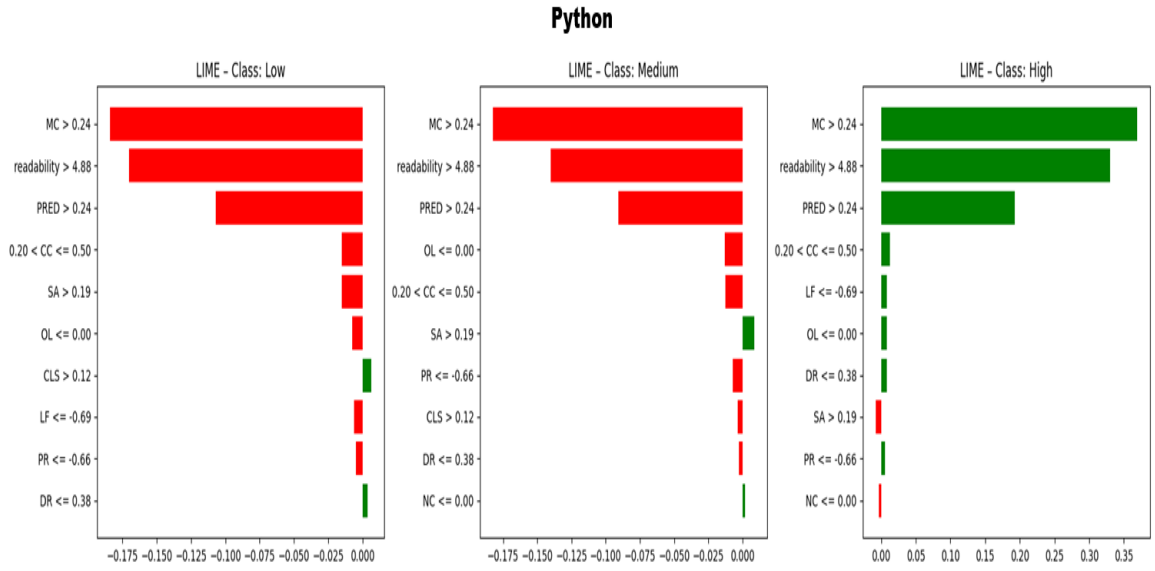

Figure 4. LIME explanations for Python snippet-level predictions (Study 2): MC and readability score are the dominant features, consistent with SHAP findings in Study 1.

6.3 Study 3: EESQA-DELMOA - Empirical Evaluation of Software Quality Assessment through Developer Experience Level Using Metaheuristic Optimisation Algorithms

The input is a developer profile with 26 features derived from observable software development activity. Four stages follow.

Stage 1

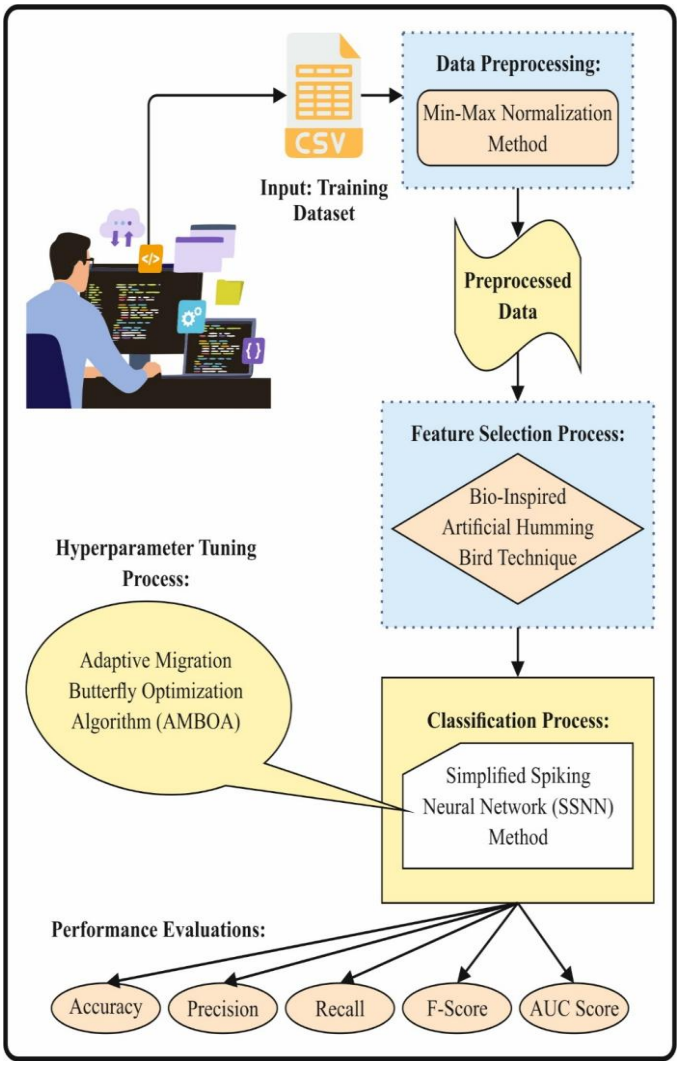

Figure 5. EESQA-DELMOA workflow: developer dataset passes through Min-Max normalisation, BAHB feature selection, SSNN classification, and AMBOA hyperparameter tuning.

Min-max normalisation. The 26 developer activity features - spanning contribution frequency, code ownership, project breadth, review participation, long-term and short-term experience metrics, and activity continuity - are normalised to the [0, 1] range: x_norm = (x − x_min) / (x_max − x_min). This removes magnitude bias from features with widely different natural scales (e.g., total commit count in the thousands versus average lines per commit in the tens), ensuring that no single feature dominates

distance-based or gradient-based computations due to scale rather than predictive value.

Stage 2
Feature selection via BAHB. The Bio-inspired Artificial Hummingbird Behaviour (BAHB) algorithm selects 18 of the 26 available features by simulating three foraging strategies: (i) Guided food search using axial, diagonal, and omnidirectional flight patterns to explore the feature space across multiple dimensions simultaneously, controlled by a directional switch vector; (ii) Territorial search in which hummingbirds intensify local search around high-quality solutions; (iii) Migration toward food sources with the lowest nectar-refilling rate, preventing convergence to suboptimal subsets. A fitness function balances classification error rate (minimised) and subset cardinality (minimised), driving selection toward compact subsets with maximum discriminative power. The 18 selected features are passed to the SSNN; the remaining 8 are discarded as redundant or noisy.

Stage 3
SSNN classification. The Simplified Spiking Neural Network classifies each developer profile into one of six categories: ESE, SA, SE, NSE, BOT, or UNK. Each arriving spike at input neuron i increases the neuron's membrane potential V_m by the corresponding synaptic weight w_i during the non-refractory period. Between spikes, V_m decays according to the leaky integrate-and-fire rule: $V_m(t) = V_m(t-1)\cdot\delta + \Sigma(w_i \cdot s_i(t))$, where δ is the decay constant and s_i(t) is the spike indicator at time t. When V_m exceeds firing threshold θ, the neuron emits a spike and resets. The network produces nonlinear responses through fine-tuned weighting of prior-layer inputs despite the linear per-neuron model. Classification is determined by the output-layer spike-rate pattern across a fixed 25-time-step observation window. The SSNN achieves 43% lower execution time than an equivalent ANN, making it viable for real-time developer classification in project management workflows.

Stage 4
AMBOA hyperparameter tuning. The Adaptive Migration Butterfly Optimisation Algorithm tunes the SSNN's hyperparameters - learning rate, decay constant δ, firing threshold θ, and network width - by treating each candidate configuration as a butterfly position in the search space. Perceived scent intensity is computed as $I_b = s \cdot P^a$, where s is stimulus intensity, P is sensory modality, and a is the power exponent. Butterflies execute alternating global search phases (PSO-inspired velocity update toward the population best, scaled by inertia weight) and local search phases (neighbourhood perturbation). A linearly decaying inertia weight $\omega(t) = \omega_{max} - (\omega_{max} - \omega_{min}) \cdot t / t_{max}$ progressively shifts the algorithm from global exploration in early iterations to local exploitation in later ones, preventing premature convergence. Validation accuracy serves as the fitness function, ensuring selected hyperparameters generalise to unseen developer profiles.

7. MAJOR CONTRIBUTIONS

The thesis makes the following original contributions to the field of program comprehension and software quality assessment:

Contribution 1
A ten-dimensional, linguistically and cognitively grounded readability parameter set for source code identifiers. The ten features - MC, NC, OL, DR, PR, LF, CC, SA, CLS, and PRED - provide a richer representation of naming quality than any previously published set, capturing semantic, structural, contextual, and cognitive dimensions simultaneously.

Contribution 2
The IRAF-XADL framework, which is the first published system to combine language-specific AST-based identifier extraction, CodeBERT contextual embeddings, and a Self-Attention BiLSTM classifier for identifier readability assessment across both Python and C++. The framework achieves state-of-the-art accuracy and is accompanied by SHAP-based explanations at both global and local levels, making its predictions auditable.

Contribution 3
The ECRVR-MVEL model, which introduces weighted majority voting over three structurally diverse deep classifiers (GCN, DBN, Bi-TCN) for snippet-level code readability prediction. The ensemble design improves robustness compared to any single classifier in the combination. LIME-based local explanations make the model's decisions interpretable to practitioners.

Contribution 4
The EESQA-DELMOA system, which demonstrates that developer experience level can be classified from observable activity features with high accuracy using a Simplified Spiking Neural Network tuned by a metaheuristic optimisation algorithm. The system achieves this with an execution time (8.27 seconds) substantially lower than all compared baselines, indicating practical viability for deployment in project management workflows.

Contribution 5
A multi-level, explainable program comprehension framework that links assessment at the identifier level, the snippet level, and the developer level under a shared commitment to interpretability. The three studies share a dataset for code-level analysis (Code Snippets: Insights and Readability, Kaggle) and use an independent dataset for developer-level analysis (Perez et al., 2023, Zenodo), establishing reproducibility.

8. RESULTS AND DISCUSSION

8.1 Dataset Description

Studies 1 and 2 use the Code Snippets: Insights and Readability dataset (Kaggle: https://www.kaggle.com/datasets/paakhim10/code-snippets-insights-and-readability/data). The dataset contains Python and C++ code snippets annotated with readability levels. For Python: 560 High, 560 Medium, 561 Low (1,681 total). For C++: 502 High, 500 Medium, 502 Low (1,504 total).

Study 3 uses the developer experience dataset published by Perez, Urtado, and Vauttier (2023) at Zenodo (https://zenodo.org/records/7011334). The dataset contains 703 developer profiles across six classes: ESE (69), SA (29), SE (73), NSE (17), BOT (10), and UNK (505). A total of 26 features are available; 18 are selected by the BAHB algorithm.

8.2 Performance of IRAF-XADL (Study 1)

All results are reported on 70%/30% train/test splits.

Python dataset: Training accuracy 98.13% (precision 97.22%, recall 97.20%, F1 97.21%, AUC 97.90%). Test accuracy 97.36% (precision 96.14%, recall 96.01%, F1 96.03%, AUC 97.02%).

C++ dataset: Training accuracy 98.42% (precision 97.62%, recall 97.61%, F1 97.61%, AUC 98.21%). Test accuracy 97.94% (precision 96.96%, recall 96.85%, F1 96.89%, AUC 97.64%).

Comparative analysis against seven baseline methods demonstrates the margin achieved by IRAF-XADL. On Python, the best-performing baseline is SMO at 82.00% accuracy; IRAF-XADL reaches 98.13%. On C++, MLP achieves 79.33%; IRAF-XADL reaches 98.42%. The gain is consistent across all five metrics.

SHAP analysis reveals a consistent finding across both languages: Meaningful Clarity (MC) and Naming Conformance (NC) are the dominant predictors of identifier readability. The composite readability score, by contrast, contributes minimally - a result that validates the decision to build ten purpose-designed features rather than rely on an aggregate metric.

8.3 Performance of ECRVR-MVEL (Study 2)

Python dataset, 30% test split: ECRVR-MVEL achieves 98.15% accuracy, 97.23% precision, 97.24% recall, 97.21% F1. The ensemble substantially outperforms each of its constituent classifiers: GCN achieves 92.87%, DBN 94.46%, and Bi-TCN 95.38% on the same split. The best non-ensemble baseline (Neural Network) reaches 90.11%; ECRVR-MVEL exceeds it by more than eight percentage points.

C++ dataset, 30% test split: ECRVR-MVEL achieves 98.38% accuracy, 97.61% precision, 97.60% recall, 97.59% F1. All baselines (Decision Tree 92.84%, Logistic Regression 69.23%, Naïve Bayes 94.58%, Neural Network 88.58%) are clearly surpassed.

LIME explanations show that Meaningful Clarity (MC) and the raw readability score are the features

most frequently identified as influential for Low and High class predictions, aligning with the SHAP findings from Study 1 and reinforcing the validity of the readability parameter set across both levels of analysis.

8.4 Performance of EESQA-DELMOA (Study 3)

70% training split: Average accuracy 98.10%, precision 93.84%, recall 75.21%, F1 79.54%, AUC 86.65%.

30% test split: Average accuracy 98.74%, precision 96.16%, recall 83.54%, F1 85.38%, AUC 90.84%.

The execution time of 8.27 seconds compares favorably with all baselines: Random Forest (14.57s), Decision Tree (16.18s), CNN (17.33s), AlexNet (15.82s). This efficiency gap is relevant to practical deployment, where real-time or near-real-time classification of developer profiles may be required.

Against seven baselines on accuracy, EESQA-DELMOA (98.74%) exceeds the next best (CNN at 94.78%) by approximately four percentage points. The margin on precision (96.16% vs 95.60% for Naïve Bayes) is narrower; the recall gap reflects the class imbalance in the dataset, particularly the small BOT and NSE classes.

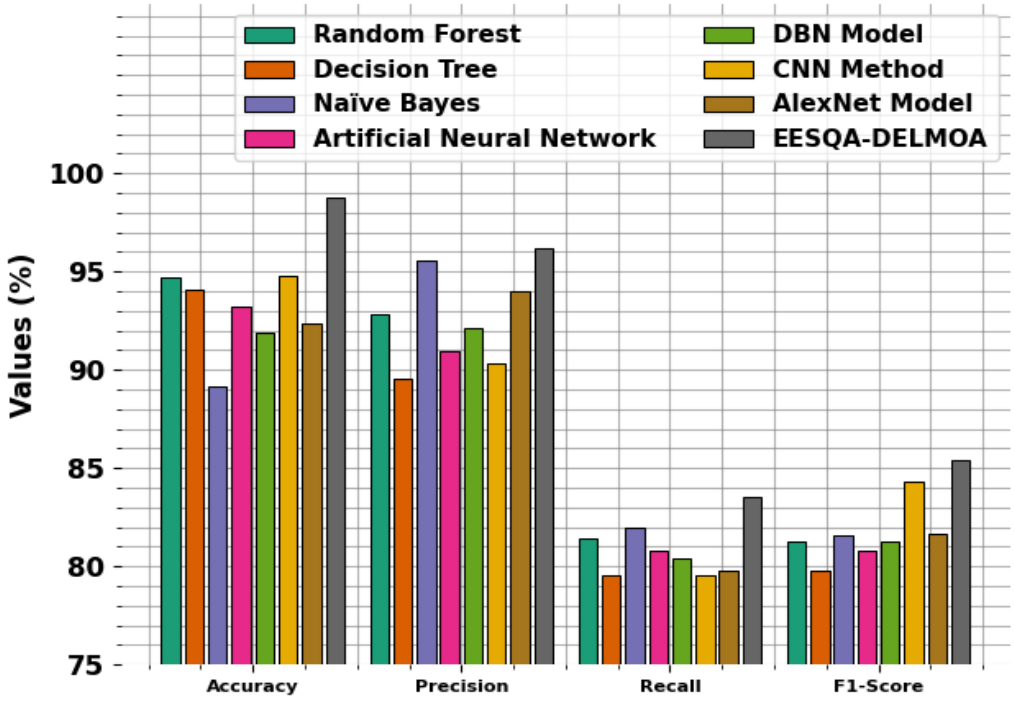

Figure 6. EESQA-DELMOA versus baseline methods: accuracy, precision, recall, and F1-score. EESQA-DELMOA reaches 98.74% accuracy, outperforming all compared methods.

8.5 Cross-Study Observations

A consistent finding emerges across all three studies: explainability components do not merely annotate results - they validate modelling choices. SHAP in Study 1 and LIME in Study 2 both identify Meaningful Clarity and Naming Conformance as the primary signals for readability assessment. This coherence across two independent explainability methods and two levels of analysis provides evidence that the feature engineering choices in Study 1 generalise beyond the identifier level. The features are not tuned to a single task; they reflect genuine properties of readable code.

The accuracy progression from identifiers (97.36%) to snippets (98.15%) to developer profiles (98.74%) does not imply that the developer level is the easiest problem - the datasets and task definitions differ. It does confirm that deep learning methods with appropriate architecture choices and explainability integration are effective across all three levels.

9. SUMMARY AND CONCLUSIONS

This thesis set out to address a fundamental limitation of existing automated code quality tools: their restriction to a single level of analysis, and their lack of interpretable outputs. Three studies were conducted, each targeting a different level of abstraction in the program comprehension hierarchy.

Study 1 (IRAF-XADL) introduced a ten-feature readability parameter set for individual identifiers and demonstrated that combining these features with CodeBERT embeddings and a Self-Attention BiLSTM produces classifiers that substantially outperform all published baselines on both Python and C++ data. The SHAP analysis identified MC and NC as the dominant predictors - a practically useful finding, since it tells developers precisely which dimension of naming quality matters most for automated assessment.

Study 2 (ECRVR-MVEL) showed that snippet-level readability prediction benefits from an ensemble of structurally diverse classifiers. Neither GCN, DBN, nor Bi-TCN individually matches the performance of their weighted combination, supporting the theoretical expectation that ensemble diversity reduces variance. LIME explanations confirmed the relevance of the readability parameters first proposed in Study 1, suggesting that the feature set generalises beyond the identifier level.

Study 3 (EESQA-DELMOA) demonstrated that developer experience level can be classified from observable activity features with high accuracy and low latency using a Simplified Spiking Neural Network tuned by a metaheuristic algorithm. The result is practically significant: a tool that classifies developer experience in 8.27 seconds could realistically support project assignment and code review pairing decisions.

Taken together, the three studies establish that program comprehension can be assessed automatically, accurately, and interpretably at every level of the software development artefact hierarchy - from the individual identifier name, through the code snippet, to the human who wrote the code. No single prior work has addressed all three levels under a unified commitment to explainability.

Limitations and future directions. The current framework is limited to Python and C++ for code-level studies. Extension to additional languages, particularly Java and JavaScript, is straightforward given the availability of AST parsers. The developer experience study relies on a dataset in which one class (Unknown) accounts for 72% of instances; improved class balance through data collection or oversampling would likely improve recall on minority classes. Fine-tuning CodeBERT rather than using it as a frozen feature extractor is expected to improve performance further, at the cost of substantially increased training time. Integration of all three levels into a unified pipeline - where identifier-level scores feed into snippet-level assessment, which in turn informs developer-level profiling - is the most natural extension of this work.

10. LIST OF REFERENCES

1. Lawrie, Dawn, Christopher Morrell, Henry Field, and David Binkley. "What's in a Name? A Study of Identifiers." In *Proceedings of the 14th IEEE International Conference on Program Comprehension*, 3–12. Athens, Greece, 2006.

2. Buse, Raymond P. L., and Westley R. Weimer. "A Metric for Software Readability." In *Proceedings of the 2008 International Symposium on Software Testing and Analysis*, 121–130. New York: ACM, 2008.

3. Feng, Zhangyin, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. "CodeBERT: A Pre-Trained Model for Programming and Natural Language." In *Findings of EMNLP 2020*, 1536–1547. Online: Association for Computational Linguistics, 2020.

4. Lundberg, Scott M., and Su-In Lee. "A Unified Approach to Interpreting Model Predictions." In *Advances in Neural Information Processing Systems 30*, edited by I. Guyon et al., 4765–4774. Red Hook, NY: Curran Associates, 2017.

5. Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. "'Why Should I Trust You?': Explaining the Predictions of Any Classifier." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144. New York: ACM, 2016.

6. Perez, Quentin, Christelle Urtado, and Sylvain Vauttier. "Dataset of Open-Source Software Developers Labeled by Their Experience Level in the Project and Their Associated Software Metrics." *Data in Brief* 46 (2023): 108842.

7. Mi, Qing, Zheng Xiao, Yu Zhan, Lingjie Tao, and Jingyi Zhang. "Towards Explainable Code Readability Classification With Graph Neural Networks." *Journal of Software: Evolution and Process* 37, no. 9 (2025): e70048.

8. Scalabrino, Simone, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. "Automatically Assessing Code Understandability." *IEEE Transactions on Software Engineering* 45, no. 10 (2019): 1012–1031.

9. Tokumoto, Satoshi, Shinji Kusumoto, and Ryohei Imai. "Development and Evaluation of a Deep Learning-Based Model for Source Code Quality Classification Using Industrial Data." *Journal of Software Engineering Practice* 6, no. 1 (2025): 1–19.

10. Salamea, María José, and Carles Farré. "Influence of Developer Factors on Code Quality: A Data Study." In *Proceedings of the 2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion*, 120–125. Sofia, Bulgaria: IEEE, 2019.

11. Yadav, Ankit, Sandeep K. Singh, and Jasjit S. Suri. "Ranking of Software Developers Based on Expertise Score for Bug Triaging." *Information and Software Technology* 112 (2019): 1–17.

12. Garousi, Vahid, Ayse Tarhan, Dietmar Pfahl, Ahmet Coşkunçay, and Onur Demirörs. "Correlation of Critical Success Factors with Success of Software Projects: An Empirical Investigation." *Software Quality Journal* 27 (2019): 429–493.

11. Journal Publication

1. B. B. Mane and R. Achary, "Evaluating Identifier Readability Using CodeBERT Embeddings and Self-Attention Bi-LSTM with Explainable Modeling", *Engineering, Technology & Applied* Science Research, vol. 16, no. 3, pp. 36731–36737, June, 2026.

2. B. B. Mane and R. Achary, "Explainable Artificial Intelligence with Hybrid Ensemble Learning based Automated Code Comprehension Prediction", *Engineering, Technology & Applied Science Research*, vol. 16, no. 4, pp. 37326–37331, August, 2026.

3. B. B. Mane and R. Achary, "Feature Optimization with Simplified Spiking Neural Network for Developer-Centric Software Quality Assessment", Engineering, Technology & Applied Science Research, 2026. *(Accepted, in production - vol., no., and pp. pending)*